Porting ICD to the Pascaline Graphics/Windowing Libraries

Feasibility Report

Assessment of converting the ICD CAD program (SVS Pascal, 1991–92)
to Pascaline / Pascal-P6 with the `graphics` and `windows` standard libraries

July 2026

1 Verdict

The port is very feasible, and easier than it first looks. The reason is structural: ICD is already built as clean layers, and the entire bottom half of its stack — the part that took assembly language and card-specific drivers — is exactly what the Pascaline `graphics` / `windows` libraries (implemented over Ami) now provide. What survives the port is the top half: the viewport math, the display-list redraw, the button autoarranger, and the command logic — and those are portable ISO 7185 Pascal that was written to be portable in the first place. The `video` directory even contains Pascal reference implementations of every assembly primitive, which is evidence the algorithms never depended on the hardware.

There is exactly one architectural change the port forces (save-under rubber-banding replaced by xor rubber-banding), a handful of small shims, and a lot of straightforward deletion.

The intent of the port is *not* to move ICD onto the Pascaline `widgets` library. ICD keeps its own look: a series of windows and text-captioned buttons, drawn by ICD itself, with its own automatic menu-arrangement logic — optionally later formed from frameless child windows as building blocks.

2 What gets deleted outright (≈40% of the graphics code)

ICD component	Fate
`pixel.asm` card drivers (vga / vesa / wd / tseng / ati / oti / trident, bank switching)	Gone — Ami owns the display
`drawa.asm` / `drawb.asm` speed primitives and `pl` / `pk` dispatch	Gone — `line`, `frect`, `setpixel` in `graphics`
`iniscn` / `resscn`, driver label selection (`tseng1024x768x16`)	Gone — library initializes; resolution via `maxxg` / `maxyg`
`inialpha` / `iniwidth` bitmap font and width tables (`drawb.pas`, `drawc.pas`)	Gone — fonts, `strsiz`, `chrpos`, `baseline` in `graphics`
`ibmrot.asm` (`kbdrdy` / `kbdinp` / `gettim`, interrupt hooks)	Gone — `etchar` events; timers via `timer()` / `ettim`; `services` for time
`position/` (Summasketch serial driver, MS mouse, `updptr` polling)	Gone — `etmoumov` / `etmoumovg` / `etmouba` / `etmoubd` events. The tablet abstraction collapses to the mouse

Screen-blank idle logic, `.pif` files, DOS `intdos` calls	Gone (`services` covers directory listing for `getlst` / `fillst`)

3 What ports with mechanical substitution

Drawing calls. `line(vp,...)` bottoms out in `graphics.line`; `block` → `frect`; `frame` → `rect` (or stays as four lines); `pier` → `fellipse` / `farc`; `dogrid` → `setpixel` loop, unchanged.

Viewport layer (`icdb.pas`). `viewc` / `realc` / `scndist` / `realdist` and the clipper are pure Pascal. Since Ami now implements viewports and scaling natively, there is a choice: keep ICD's own transform layer untouched (lowest-risk, works immediately) or migrate to Ami's viewports later as a cleanup. The recommendation is to port with ICD's own layer first — it is known-good and makes the first bring-up a near-transliteration.

Colors. ICD's 16-color EGA enumeration does not match the 8-color `terminal` enumeration, but `fcolorg(r, g, b)` reproduces all 16 exactly — one small mapping function.

Vector text. `vchar`, with rotation and scaling for schematic labels, is already implemented as line strokes over `line()`, so it ports as-is. Alternatively font 4 (the technical font) covers the unrotated cases.

Redraw model. ICD already redraws entirely from its display list (`redraw` / `dispsht` / `drwfig`). That is precisely what unbuffered `windows` mode wants — `etredraw` arrives, ICD calls `redraw`. Or run buffered mode and get occlusion repair for free. ICD's model is *ahead* of what the library requires; nothing to write here.

Buttons and the autoarranger. `butrec`, `plcbut` / `updbut`, the active / disabled / alert states, `edtbut` in-button editing, and the `arrbutt` / `marginr` / `arrbutr` / `tfit` fitting loop are all pure Pascal operating on rectangles and strings. They port essentially unchanged, with two substitutions: the pixel-width table lookups become `strsiz` calls (which actually *improves* the fit calculation, since kerning is accounted for), and hit-testing keeps using the same `region` records against mouse-event coordinates. Keeping the ICD look — beveled frames, top and right button margins, text-captioned buttons — is just the same `block` / `line` / `plcstr` sequences on the new primitives. If later each button or the frame should become a genuine window, the "windows as building blocks" technique (Pascaline report M.20, frameless child windows) supports exactly that, but nothing forces it for the first port.

4 The one real design change: rubber-banding

ICD's transient graphics (`setcur` / `rescur`, `setline`, `setbox`, `setcircle`, `setarc`, `linesav` / `linerst`, `boxsav` / `boxrst`, `arcsavc` / `arcrstc`) all work by **save-under** — reading pixels with `getpix` into a `linarr` buffer and restoring them. The Pascaline `graphics` library has no pixel read-back (deliberately, per its remote-display design). The replacement is the library's intended mechanism: **xor mode** (`fxor`), where drawing the same figure twice removes it. This is a simplification, not a workaround — every `set`X/`res`X pair becomes "draw figure in xor" / "draw same figure in xor again," and the save buffers, indices, and the whole `linesav` / `linerst` / `arcsavc` / `arcrstc` machinery disappear.

Two cautions from the specification that ICD must respect: corner pixels of composite figures

double-xor (draw rubber boxes as four non-overlapping lines, or accept corner sparkle since ICD's marks are simple), and erase must use identical parameters to draw — which ICD's paired set/res structure already guarantees. The crosshair cursor can either stay as an xor-drawn crosshair or simply yield to the system mouse pointer.

Related small shim: ICD's arcs are specified by start / end / center points; `graphics.arc` takes a bounding box plus angle ratios (0..maxint = 0..360°). ICD already computes `angle()` — converting to the ratio form is a few lines. Alternatively ICD's own Pascal arc rasterizer keeps working over `setpixel` if pixel-identical arcs matter for the first pass.

5 Main-loop restructuring

This is the second-largest work item after rubber-banding, but it is shape-changing rather than logic-changing. ICD's core is a poll loop (`updptr`, `kbdrdy`, cursor tracking, `chkbut`); Pascaline is event-driven. The conversion is the classic one: the poll loop becomes an `event(er)` loop dispatching `etchar` → the keyboard and edit paths, `etmoumovg` → `movcur`, `etmouba` / `etmoubd` → the puck-button logic in `updbut` / `drag`, `ettim` → the elapsed / wait uses, and `etterm` → exit. Since Pascal-P6 does not yet implement objects, the procedural event-record interface is used, not the `ev...` callback overrides or the `screen` / `surface` classes — ICD is procedural anyway, so this costs nothing. `wait()` becomes a timer event rather than a busy loop.

6 Language-level friction (minor)

- SVS `module` / `uses {$U common.j}` maps directly onto Pascaline modules; `define.pas` / `common.pas` become a proper shared module, and the dozens of per-file `external` redeclarations disappear in favor of real cross-module visibility — a large readability win.
- `integer = longint` / `real = double` redefinitions: drop them; Pascaline integers are natural-size and `lreal` exists if needed.
- `cexternal` vanishes as a concept.
- Watch for identifiers that are now reserved words (`out`, `view`, `fixed`, `class` are the likely suspects in 700 KB of code). A compile pass will flush any collisions.
- The printer path (`fjdl3400`, `pagprt` / `strprt` / `outbuf`) rewrites against the `graphics` printer / page-buffer model via the `list` file — a genuine rewrite, but small and isolated in `icde.pas`.

7 Effort estimate

Roughly in order:

1. Mechanical module restructuring and compile-clean under Pascal-P6 — large in line count, trivial in thought.

2. Event-loop conversion — a few days of careful work in `icdc.pas`'s `updpuck` / `movcur` region.
3. Xor rubber-banding — systematic replacement across roughly 16 set/res pairs in `icdb.pas`, each one getting *shorter*.
4. Font and metrics substitution in the button and text code.
5. Printer path.

Nothing in the port needs objects, matrix math, or widgets, so Pascal-P6's current gaps do not block it. The highest-risk item is subtle behavioral differences in xor rubber-banding appearance versus save-under; everything else is substitution with existing, working ICD logic riding on top.

Net: this is a port, not a rewrite — the parts of ICD that were hard to write (viewport math, display-list redraw, menu auto-layout, command semantics) transfer intact, and the parts that do not transfer are the parts Ami was built to replace.